Lista dwukierunkowa

Szablon deque jest to struktura danych reprezentująca dwukierunkową listę ze swobodnym dostępem do wszystkich elementów przechowywanych w kontenerze. Typ przechowywanych elementów określa parametr szablonu Type.

Szablon deque umożliwia dodawanie oraz usuwanie elementów na początku i końcu kolejki w czasie stałym - O(1). Dodawanie oraz usuwanie elementów w innych miejscach niż początek oraz koniec kontenera możliwe jest w czasie liniowym - O(n), gdzie n to liczba elementów w kontenerze.

Składnia

```
#include <deque>
using namespace std;

{

template <
class Type,
class Allocator = allocator < Type >
>
class deque;

}
```

- class Type - Typ danych jakiego mają być elementy, które są przechowywane przez kontener.
- class Allocator - Alokator, który jest odpowiedzialny za zarządzanie pamięcią. Domyślnie używany jest standardowy alokator pamięci, tj. std::allocator<Type>.

Metody

- `assign` Usuwa wszystkie istniejące elementy z kontenera, a następnie kopiuje wskazane elementy do kontenera.
- `at` Zwraca referencję na element, który znajduje się na podanej pozycji w kontenerze
- `back` Zwraca referencję na ostatni element w kontenerze.
- `begin` Zwraca iterator wskazujący na pierwszy element.
- `clear` Usuwa wszystkie elementy z kontenera
- `empty` Sprawdza czy kontener jest pusty.
- `end` Zwraca iterator wskazujący na element będący za ostatnim elementem.
- `erase` Usuwa jeden element lub wiele elementów z kontenera występujących na podanej pozycji lub w podanym zakresie.
- `front` Zwraca referencję na pierwszy element w kontenerze.
- `insert` Wstawia jeden element lub wiele elementów do kontenera na określonej pozycji.
- `max_size` Maksymalna możliwa liczba elementów w kontenerze
- `pop_back` Usuwa jeden element z kontenera, znajdujący się na jego końcu.
- `pop_front` Usuwa jeden element z kontenera znajdujący się na jego początku.
- `push_back` Dodaje nowy element na końcu kontenera
- `push_front` Dodaje nowy element na początku kontenera
- `rbegin` Zwraca iterator odwrotny wskazujący na ostatni element.

- `rend` Zwraca iterator odwrotny wskazujący na element występujący bezpośrednio przed pierwszym elementem.
- `resize` Ustawia nowy rozmiar kontenera deque.
- `size` Zwraca liczbę elementów znajdujących się aktualnie w kontenerze.
- `swap` Kontenery zamieniają się posiadanymi danymi.

assign

Przypisuje nową zawartość do kontenera deque, zastępując jego obecną zawartość i modyfikując odpowiednio jego rozmiar.

```
1  #include<iostream>
2  #include<deque>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE, "Polish");
7
8  deque<int> k1;
9  deque<int> k2;
10 deque<int> k3;
11
12 k1.assign(5,9); // wstawienie 5 int o wartości 9
13
14 deque<int> ::iterator it;
15 it=k1.begin()+1;
16 k2.assign(it, k1.end()-1); // 3 środkowe wartości
17
18 int tab[4]={1,1,2,2};
19 k3.assign(tab,tab+4);
20
21 cout<<"rozmiar k1="<<k1.size()<<endl; // 5
22 cout<<"rozmiar k2="<<k2.size()<<endl; // 3
23 cout<<"rozmiar k3="<<k3.size()<<endl; // 4
24
25 return 0;}
```

at

Zwraca referencję do elementu na pozycji n w obiekcie kontenera deque. Funkcja automatycznie sprawdza, czy n mieści się w granicach prawidłowych elementów w kontenerze, zwracając wyjątek out_of_range, jeśli nie.

```
1  #include<iostream>
2  #include<deque>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE, "Polish");
7
8  deque<int> k1 (10); // 10 wartości 0 (domyślne)
9  for (unsigned i=0; i<k1.size(); i++){
10     k1.at(i)=i;}
11
12 cout << "deque :";
13 for (unsigned i=0; i<k1.size(); i++){
14     cout << ' ' << k1.at(i);}
15
16 // deque : 0 1 2 3 4 5 6 7 8 9
17 return 0;}
```

back

Odwołanie do ostatniego elementu w kontenerze deque.

```
1  #include<iostream>
2  #include<deque>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE, "Polish");
7
8  deque<int> k1;
9
10 k1.push_back(10);
11 k1.push_back(20);
12
13 cout<<"k1.back()="<<k1.back(); // 20
14
15 return 0;}
```

front

Odwołanie do pierwszego elementu w kontenerze deque.

```
1  #include<iostream>
2  #include<deque>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE,"Polish");
7
8  deque<int> k1;
9  k1.push_back(10);
10 k1.push_back(20);
11 k1.push_back(30);
12 cout<<"k1.front()="<<k1.front(); // 10
13 return 0;
14 }
```

erase

Usuwa z kontenera deque pojedynczy element (pozycję) lub zakres elementów ([pierwszy, ostatni)).

```
1  #include<iostream>
2  #include<deque>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE, "Polish");
7
8  deque<int> k1;
9  for (int i=1; i<=10; i++) {k1.push_back(i);}
10
11 //usunięcie 6 elementu
12 k1.erase (k1.begin()+5);
13
14 //usunięcie pierwszych 3 elementów
15
16 k1.erase (k1.begin(),k1.begin()+3);
17 return 0;}
```

begin

Zwraca iterator wskazujący na pierwszy element w kontenerze deque.

```
1  #include<iostream>
2  #include<deque>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE,"Polish");
7
8  deque<int> k1;
9  deque<int>::iterator it;
10 it=k1.begin();
11 for (int i=2; i<=5; i++) k1.push_back(i);
12 // 2 3 4 5
13 cout<<*it;
14 // 2
15 return 0;
16 }
```

end

Zwraca iterator wskazujący na ostatni element w kontenerze deque.

```
1  #include<iostream>
2  #include<deque>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE,"Polish");
7
8  deque<int> k1;
9  deque<int>::iterator it;
10
11 for (int i=2; i<=5; i++) k1.push_back(i);
12 // 2 3 4 5
13 it=k1.end();
14 it--;
15 cout<<*it;
16 // 5
17
18 return 0;
19 }
```

rbegin

Zwraca odwrócony iterator wskazujący na ostatni element w kontenerze (czyli jego odwrócone początkowanie). Iteratory odwrócone iterują wstecz: zwiększanie ich przesuwa je w kierunku początku kontenera.

```
1  #include<iostream>
2  #include<deque>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE,"Polish");
7
8  deque<int> k1;
9  deque<int>::reverse_iterator rit;
10
11 for (int i=2; i<=5; i++) k1.push_back(i); // 2 3 4 5
12 rit=k1.rbegin();
13 cout<<*rit; // 5
14 rit++;
15 cout<<*rit; // 4
16 return 0;
17 }
```

rend

Zwraca odwrócony iterator wskazujący na pierwszy element w kontenerze (czyli jego odwrócone początkowanie). Iteratory odwrócone iterują wstecz: zwiększanie ich przesuwa je w kierunku początku kontenera.

```
1  #include<iostream>
2  #include<deque>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE,"Polish");
7
8  deque<int> k1;
9  deque<int>::reverse_iterator rit;
10
11 for (int i=2; i<=5; i++) k1.push_back(i); // 2 3 4 5
12 rit=k1.rend()-1;
13 cout<<*rit; // 2
14 rit--;
15 cout<<*rit; // 3
16 return 0;
17 }
```

size

Zwraca liczbę elementów w kontenerze.

```
1  #include<iostream>
2  #include<deque>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE,"Polish");
7
8  deque<int> k1;
9   cout<<"k1.size()="<<k1.size()<<endl;   // 0
10  k1.push_back(10);
11  k1.push_back(10);
12  k1.push_back(10);
13  cout<<"k1.size()="<<k1.size()<<endl;   // 3
14 return 0;
15 }
```

resize

Zmienia rozmiar kontenera tak, aby zawierał n elementów. Jeśli n jest mniejsze niż obecny rozmiar kontenera, zawartość zostaje zmniejszona do pierwszych n elementów, usuwając te poza nimi. Jeśli n jest większe niż aktualny rozmiar kontenera, zawartość jest rozszerzana poprzez wstawienie na końcu tylu elementów, ile jest potrzebne, aby osiągnąć rozmiar n.

```
1  #include<iostream>
2  #include<deque>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE,"Polish");
7
8  deque<int> k1;
9  deque<int>::iterator it;
10
11  for (int i=1; i<10; ++i){ k1.push_back(i);} // k1={1,2,3,4,5,6,7,8,9}
12
13   for (it = k1.begin(); it != k1.end(); ++it){cout << ' ' << *it;}cout<<endl;
14   // k1={1,2,3,4,5,6,7,8,9}
15   k1.resize(5);
16   for (it = k1.begin(); it != k1.end(); ++it){cout << ' ' << *it;}cout<<endl;
17   // k1={1,2,3,4,5}
18   k1.resize(7,99);
19   for (it = k1.begin(); it != k1.end(); ++it){cout << ' ' << *it;}cout<<endl;
20   // k1={1,2,3,4,5,99,99}
21   k1.resize(11);
22   for (it = k1.begin(); it != k1.end(); ++it){cout << ' ' << *it;}cout<<endl;
23   // k1={1,2,3,4,5,99,99,0,0,0,0}
24 return 0;
25 }
```

max_size

Zwraca maksymalną liczbę elementów, które kontener deque może przechowywać. Jest to maksymalny potencjalny rozmiar, jaki kontener może osiągnąć ze względu na znane ograniczenia implementacji systemu lub biblioteki, ale nie ma żadnej gwarancji, że kontener będzie w stanie osiągnąć ten rozmiar: wciąż może nie udać się zaalokować pamięci w dowolnym momencie przed osiągnięciem tego rozmiaru.

```
1  #include<iostream>
2  #include<deque>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE,"Polish");
7
8  deque<int> k1;
9  int ilosc;
10 cout<<"podaj wielkosc kontenera k1:"<<endl;
11 cin>>ilosc;
12 if(ilosc<k1.max_size()) {k1.resize(ilosc,99);}
13 else {cout<<"przekroczona maksymalna wielkość kontenera deque";}
14 return 0;
15 }
```

empty

Zwraca, czy kontener deque jest pusty (czyli czy jego rozmiar wynosi 0).

```
1  #include<iostream>
2  #include<deque>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE,"Polish");
7
8  deque<int> k1;
9  if(k1.empty()){cout<<"kontener k1 jest pusty"<<endl;}  // jest pusty
10 else          {cout<<"kontener k1 NIE jest pusty"<<endl;}
11 k1.push_back(10);
12 if(k1.empty()){cout<<"kontener k1 jest pusty"<<endl;}
13 else          {cout<<"kontener k1 NIE jest pusty"<<endl;} // nie jest pusty
14
15 return 0;
16 }
```

clear

Usuwa wszystkie elementy z deque (które są niszczone), pozostawiając kontener o rozmiarze 0.

```
1  #include<iostream>
2  #include<deque>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE,"Polish");
7
8  deque<int> k1;
9  k1.push_back(10);
10 if(k1.empty()){cout<<"kontener k1 jest pusty"<<endl;}
11 else          {cout<<"kontener k1 NIE jest pusty"<<endl;}   // nie jest pusty
12 k1.clear();
13 if(k1.empty()){cout<<"kontener k1 jest pusty"<<endl;}       // jest pusty
14 else          {cout<<"kontener k1 NIE jest pusty"<<endl;}
15 cout<<"size="<<k1.size();                                    // size=0
16 return 0;
17 }
```

pop_back

Usuwa ostatni element w kontenerze deque, efektywnie zmniejszając rozmiar kontenera o jeden.

```
1  #include<iostream>
2  #include<deque>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE,"Polish");
7
8  deque<int> k1;
9  k1.push_back(10);
10 k1.push_back(20);
11 k1.push_back(30);        // 10 20 30
12 cout<<k1.back()<<endl;   // 30
13 k1.pop_back();
14 cout<<k1.back()<<endl;   // 20
15 k1.pop_back();
16 return 0;
17 }
```

pop_front

Usuwa pierws element w kontenerze deque, efektywnie zmniejszając rozmiar kontenera o jeden.

```
1  #include<iostream>
2  #include<deque>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE,"Polish");
7
8  deque<int> k1;
9  k1.push_back(10);
10 k1.push_back(20);
11 k1.push_back(30);
12 cout<<"size="<<k1.size();        // size=3
13 cout<<endl<<k1.front()<<endl;    // 10
14 k1.pop_front();
15 cout<<k1.front()<<endl;          // 20
16 k1.pop_front();
17 cout<<"size="<<k1.size();        // size=1
18 return 0;
19 }
```

insert

Kontener deque jest rozszerzany poprzez wstawianie nowych elementów przed elementem na określonej pozycji. To efektywnie zwiększa rozmiar kontenera o ilość wstawionych elementów.

Kolejki dwustronne są zaprojektowane tak, aby efektywnie wykonywać wstawianie (i usuwanie) zarówno na końcu, jak i na początku sekwencji. Wstawianie na innych pozycjach jest zazwyczaj mniej efektywne niż w kontenerach listy lub forward_list.

```
1  #include<iostream>
2  #include<deque>
3  #include<vector>
4  using namespace std;
5
6  int main()
7  {setlocale(LC_CTYPE,"Polish");
8
9  deque<int> k1;
10 for (int i=1; i<6; i++) {k1.push_back(i);}  // 1 2 3 4 5
11 deque<int>::iterator it = k1.begin();       // ^
12 it++;                                       //   ^
13 it=k1.insert(it,10);                        // 1 10 2 3 4 5
14 // it ustawiony na tq 10                        ^
15 k1.insert (it,2,20);                        // 1 20 20 10 2 3 4 5
16 it = k1.begin()+2;                          //       ^
17 vector<int> myvector (2,30);                // 30 30
18 k1.insert (it,myvector.begin(),myvector.end());
19                                             // 1 20 30 30 20 10 2 3 4 5
20
21 return 0;
22 }
```

swap

Zamienia zawartość kontenera na zawartość x, który jest innym obiektem deque zawierającym elementy tego samego typu. Rozmiary mogą się różnić. Po wywołaniu tej funkcji składowej, elementy w tym kontenerze są tymi, które były w x przed wywołaniem, a elementy x są tymi, które były w tym. Wszystkie iteratory, referencje i wskaźniki pozostają ważne dla zamienionych obiektów.

```
1  #include<iostream>
2  #include<deque>
3  #include<vector>
4  using namespace std;
5
6  int main()
7  {setlocale(LC_CTYPE,"Polish");
8
9  deque<int> k1(3,1); // 1 1 1
10 deque<int> k2(5,2); // 2 2 2 2 2
11 cout<<"k1.size="<<k1.size()<<" k2.size="<<k2.size()<<endl;
12 k1.swap(k2);
13 cout<<"k1.size="<<k1.size()<<" k2.size="<<k2.size()<<endl;
14 return 0;
15 }
```

Ćwiczenie 1

Zapoznanie się z kontenerem deque i jego podstawowymi metodami.

Napisz program w C++, który:

1. Utworzy pusty kontener deque<int>.
2. Doda trzy elementy na koniec kontenera (push_back).
3. Doda jeden element na początek kontenera (push_front).
4. Wyświetli pierwszy i ostatni element listy (front, back).
5. Wyświetli aktualny rozmiar kontenera (size).

Ćwiczenie 2

Zastosowanie metody at() oraz zrozumienie kontroli zakresu.

1. Utwórz kontener deque<int> o rozmiarze 10.
2. Wypełnij go wartościami od 0 do 9.
3. Wyświetl wszystkie elementy, korzystając z metody at().
4. Spróbuj odwołać się do elementu spoza zakresu i obsłuż wyjątek out_of_range.

Ćwiczenie 3

Praca z iteratorami begin, end, rbegin, rend.

1. Utwórz kontener deque<int> i wstaw do niego liczby od 1 do 5.
2. Wyświetl zawartość kontenera:
    a. w kolejności normalnej (iteratory zwykłe),
    b. w kolejności odwrotnej (iteratory odwrotne).
3. Wyjaśnij różnicę między end() a rend() na podstawie obserwacji działania programu.

Ćwiczenie 4

Zastosowanie metod erase, insert, resize.

1. Utwórz deque<int> zawierający liczby od 1 do 10.
2. Usuń:
    a. jeden wybrany element ze środka kontenera,
    b. zakres pierwszych trzech elementów.
3. Wstaw nowy element w środku kontenera.
4. Zmień rozmiar kontenera:
    a. zmniejsz go do 5 elementów,
    b. następnie zwiększ do 8 elementów (nowe elementy ustaw na 0).

Ćwiczenie 5

Praktyczne zastosowanie deque

Zaimplementuj prostą symulację kolejki dwustronnej:

1. Program umożliwia:
    a. dodanie elementu na początek,
    b. dodanie elementu na koniec,
    c. usunięcie elementu z początku,
    d. usunięcie elementu z końca,
    e. wyświetlenie całej zawartości kolejki.
2. Program działa w pętli sterowanej menu tekstowym.
3. Przed każdą operacją usuwania należy sprawdzić, czy kontener nie jest pusty (empty()).